

# Pacers\_2025\_W/L\_Prediction\_Impact

Lindsey Hornberger

2026-05-28

## 1. Data Preparation

```
# import data
setwd("/Users/lindseyhornberger/Desktop/My Fun Projects/nba")
nba_player_data <- read.csv("traditional.csv") # player and player_id

# 2025 season data subset
nba_player_2025 <- nba_player_data %>% filter(season == 2025)

glimpse(nba_player_2025)
```

```
## Rows: 28,240
## Columns: 30
## $ gameid    <int> 22400061, 22400061, 22400061, 22400061, 22400061, 22400061, 2...
## $ date      <chr> "2024-10-22", "2024-10-22", "2024-10-22", "2024-10-22", "2024...
## $ type      <chr> "regular", "regular", "regular", "regular", "regular", "regul...
## $ playerid  <int> 1628369, 1628401, 1627759, 201950, 1630540, 201143, 1628973, ...
## $ player    <chr> "Jayson Tatum", "Derrick White", "Jaylen Brown", "Jrue Holida...
## $ team      <chr> "BOS", "BOS", "BOS", "BOS", "NYK", "BOS", "NYK", "NYK", "NYK"...
## $ home      <chr> "BOS", "BOS", "BOS", "BOS", "BOS", "BOS", "BOS", "BOS", "BOS"...
## $ away      <chr> "NYK", "NYK", "NYK", "NYK", "NYK", "NYK", "NYK", "NYK", "NYK"...
## $ MIN       <int> 30, 27, 30, 31, 26, 26, 25, 24, 21, 35, 25, 24, 24, 34, 19, 2...
## $ PTS       <int> 37, 24, 23, 18, 22, 11, 22, 12, 11, 16, 12, 4, 10, 4, 4, 3, 2...
## $ FGM       <int> 14, 8, 7, 7, 8, 4, 9, 5, 5, 7, 4, 2, 4, 1, 2, 1, 1, 0, 1, 1, ...
## $ FGA       <int> 18, 13, 18, 9, 10, 7, 14, 9, 10, 13, 6, 3, 10, 7, 3, 10, 6, 0...
## $ FG.       <dbl> 77.8, 61.5, 38.9, 77.8, 80.0, 57.1, 64.3, 55.6, 50.0, 53.8, 6...
## $ X3PM      <int> 8, 6, 5, 4, 4, 3, 1, 1, 1, 2, 0, 0, 2, 0, 0, 1, 0, 0, 1, 1, 0...
## $ X3PA      <int> 11, 10, 9, 6, 5, 5, 2, 2, 4, 7, 2, 0, 7, 4, 0, 9, 3, 0, 2, 1,...
## $ X3P.      <dbl> 72.7, 60.0, 55.6, 66.7, 80.0, 60.0, 50.0, 50.0, 25.0, 28.6, 0...
## $ FTM       <int> 1, 2, 4, 0, 2, 0, 3, 1, 0, 0, 4, 0, 0, 2, 0, 0, 0, 0, 0, 0, 0...
## $ FTA       <int> 2, 2, 4, 0, 3, 0, 3, 1, 0, 1, 6, 0, 0, 2, 0, 0, 0, 0, 0, 0, 0...
## $ FT.       <dbl> 50.0, 100.0, 100.0, 0.0, 66.7, 0.0, 100.0, 100.0, 0.0, 0.0, 6...
## $ OREB      <int> 0, 0, 2, 2, 0, 0, 0, 0, 1, 0, 1, 3, 1, 0, 2, 1, 3, 0, 0, 0, 0...
## $ DREB      <int> 4, 3, 5, 2, 0, 3, 1, 7, 2, 0, 3, 6, 4, 5, 1, 3, 4, 4, 1, 0, 0...
## $ REB       <int> 4, 3, 7, 4, 0, 3, 1, 7, 3, 0, 4, 9, 5, 5, 3, 4, 7, 4, 1, 0, 0...
## $ AST       <int> 10, 4, 1, 4, 2, 5, 2, 3, 4, 2, 3, 1, 1, 3, 1, 4, 2, 0, 0, 0, ...
## $ STL       <int> 1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0...
## $ BLK       <int> 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 2, 0, 0, 1, 0, 0, 1, 0, 0, 0...
## $ TOV       <int> 1, 0, 1, 0, 1, 0, 4, 0, 1, 1, 1, 1, 0, 0, 0, 1, 0, 0, 0, 2, 0...
## $ PF        <int> 1, 1, 3, 2, 1, 2, 3, 1, 3, 0, 0, 0, 2, 3, 0, 2, 1, 0, 1, 0, 0...
## $ X...      <int> 26, 21, 23, 23, -18, 19, -23, -18, 0, -33, -23, -5, 6, -21, 2...
## $ win       <int> 1, 1, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1, 1, 0, 0, 0, 1...
## $ season    <int> 2025, 2025, 2025, 2025, 2025, 2025, 2025, 2025, 2025, 2025, 2...
```

```
summary(nba_player_2025)
```

```

##      gameid      date      type      playerid
## Min.   :22400001 Length:28240 Length:28240 Min.   : 2544
## 1st Qu.:22400325 Class :character Class :character 1st Qu.:1627936
## Median :22400655 Mode  :character Mode  :character Median :1630168
## Mean   :23816306      Mean   :1407570
## 3rd Qu.:22400988      3rd Qu.:1631131
## Max.   :52400211      Max.   :1642530
##      player      team      home      away
## Length:28240 Length:28240 Length:28240 Length:28240
## Class :character Class :character Class :character Class :character
## Mode  :character Mode  :character Mode  :character Mode  :character
##
##
##
##      MIN      PTS      FGM      FGA
## Min.   : 0.00 Min.   : 0.00 Min.   : 0.000 Min.   : 0.000
## 1st Qu.:15.00 1st Qu.: 4.00 1st Qu.: 1.000 1st Qu.: 4.000
## Median :24.00 Median : 9.00 Median : 3.000 Median : 7.000
## Mean   :22.57 Mean   :10.61 Mean   : 3.881 Mean   : 8.321
## 3rd Qu.:31.00 3rd Qu.:16.00 3rd Qu.: 6.000 3rd Qu.:12.000
## Max.   :53.00 Max.   :61.00 Max.   :22.000 Max.   :39.000
##      FG.      X3PM      X3PA      X3P.
## Min.   : 0.0 Min.   : 0.000 Min.   : 0.000 Min.   : 0.00
## 1st Qu.:28.6 1st Qu.: 0.000 1st Qu.: 1.000 1st Qu.: 0.00
## Median :44.4 Median : 1.000 Median : 3.000 Median :25.00
## Mean   :42.9 Mean   : 1.259 Mean   : 3.498 Mean   :26.59
## 3rd Qu.:57.1 3rd Qu.: 2.000 3rd Qu.: 5.000 3rd Qu.:50.00
## Max.  :100.0 Max.   :12.000 Max.   :20.000 Max.   :100.00
##      FTM      FTA      FT.      OREB
## Min.   : 0.000 Min.   : 0.000 Min.   : 0.00 Min.   : 0.000
## 1st Qu.: 0.000 1st Qu.: 0.000 1st Qu.: 0.00 1st Qu.: 0.000
## Median : 1.000 Median : 1.000 Median :33.30 Median : 1.000
## Mean   : 1.589 Mean   : 2.039 Mean   :41.63 Mean   : 1.039
## 3rd Qu.: 2.000 3rd Qu.: 3.000 3rd Qu.:100.00 3rd Qu.: 2.000
## Max.   :21.000 Max.   :26.000 Max.   :100.00 Max.   :13.000
##      DREB      REB      AST      STL
## Min.   : 0.000 Min.   : 0.000 Min.   : 0.000 Min.   :0.0000
## 1st Qu.: 1.000 1st Qu.: 2.000 1st Qu.: 1.000 1st Qu.:0.0000
## Median : 2.000 Median : 3.000 Median : 2.000 Median :0.0000
## Mean   : 3.076 Mean   : 4.115 Mean   : 2.459 Mean   :0.7653
## 3rd Qu.: 4.000 3rd Qu.: 6.000 3rd Qu.: 4.000 3rd Qu.:1.0000
## Max.   :23.000 Max.   :28.000 Max.   :22.000 Max.   :8.0000
##      BLK      TOV      PF      X...
## Min.   : 0.0000 Min.   : 0.00 Min.   :0.000 Min.   : -58
## 1st Qu.: 0.0000 1st Qu.: 0.00 1st Qu.:1.000 1st Qu.: -7
## Median : 0.0000 Median : 1.00 Median :2.000 Median : 0
## Mean   : 0.4566 Mean   : 1.26 Mean   :1.752 Mean   : 0
## 3rd Qu.: 1.0000 3rd Qu.: 2.00 3rd Qu.:3.000 3rd Qu.: 7
## Max.   :10.0000 Max.   :11.00 Max.   :6.000 Max.   : 51
##      win      season
## Min.   :0.0000 Min.   :2025

```

```
## 1st Qu.:0.0000 1st Qu.:2025
## Median :1.0000 Median :2025
## Mean :0.5026 Mean :2025
## 3rd Qu.:1.0000 3rd Qu.:2025
## Max. :1.0000 Max. :2025
```

```
colSums(is.na(nba_player_2025))
```

```
## gameid date type playerid player team home away
## 0 0 0 0 0 0 0 0
## MIN PTS FGM FGA FG. X3PM X3PA X3P.
## 0 0 0 0 0 0 0
## FTM FTA FT. OREB DREB REB AST STL
## 0 0 0 0 0 0 0
## BLK TOV PF X... win season
## 0 0 0 0 0 0
```

```
pacers_2025 <- nba_player_2025 %>% filter(team == "IND") %>% clean_names()

pacers_2025 <- pacers_2025 %>%
  rename(plus_minus = x,
         fg_pct = fg,
         x3p_pct = x3p,
         ft_pct = ft
  )
pacers_2025 <- pacers_2025 %>%
  mutate(
    efg = ifelse(fga == 0, NA, (fgm + 0.5 * x3pm) / fga),
    ts = ifelse((fga + 0.44 * fta) == 0, NA, pts / (2 * (fga + 0.44 * fta))),
    pm_per_min = ifelse(min == 0, NA, plus_minus / min)
  )
```

## 2. Build Game-Level Impact Scores

```
pacers_2025 <- pacers_2025 %>%
  mutate(
    efg = ifelse(fga == 0, NA, (fgm + 0.5 * x3pm) / fga),
    ts = ifelse((fga + 0.44 * fta) == 0, NA, pts / (2 * (fga + 0.44 * fta))),
    pm_per_min = ifelse(min == 0, NA, plus_minus / min)
  )

player_impact <- pacers_2025 %>%
  group_by(player) %>%
  summarise(
    games = n(),
    mpg = mean(min, na.rm = TRUE),
    ppg = mean(pts, na.rm = TRUE),
    apg = mean(ast, na.rm = TRUE),
    rpg = mean(reb, na.rm = TRUE),
    efg = mean(efg, na.rm = TRUE),
    pm_per_min = mean(pm_per_min, na.rm = TRUE)
  ) %>%
  filter(games >= 10, mpg >= 10)

player_impact
```

```
## # A tibble: 12 × 8
##   player      games  mpg   ppg   apg   rpg   efg pm_per_min
##   <chr>      <int> <dbl> <dbl> <dbl> <dbl> <dbl>      <dbl>
## 1 Aaron Nesmith      68  26.1  12.2  1.18  4.53 0.596      0.150
## 2 Andrew Nembhard    88  30.1  10.7  4.92  3.30 0.532      0.0806
## 3 Ben Sheppard      84  18.1   4.75  1.11  2.57 0.545     -0.116
## 4 Bennedict Mathurin  94  26.9  14.9  1.65  4.85 0.507     -0.0165
## 5 Jarace Walker      87  15.0   5.71  1.36  2.87 0.563     -0.232
## 6 Myles Turner      95  30.0  15.1  1.51  6.12 0.575      0.0592
## 7 Obi Toppin     102  19.5  10.3  1.55  3.98 0.593     -0.0588
## 8 Pascal Siakam     101  32.9  20.3  3.38  6.79 0.568      0.124
## 9 Quenton Jackson    28  13.6   5.79  1.89  1.57 0.469     -0.230
## 10 T.J. McConnell   102  17.8   9.20  4.35  2.63 0.530     -0.0855
## 11 Thomas Bryant     76  13.4   5.79  0.697  3.24 0.552     -0.190
## 12 Tyrese Haliburton  96  33.6  18.3  9.06  3.97 0.551      0.0975
```

```
safe_scale <- function(x) {
  if (sd(x, na.rm = TRUE) == 0) return(rep(0, length(x)))
  (x - mean(x, na.rm = TRUE)) / sd(x, na.rm = TRUE)
}
```

```
player_impact <- player_impact %>%
  mutate(
    impact_score =
      safe_scale(ppg) * 0.25 +
      safe_scale(apg) * 0.20 +
      safe_scale(rpg) * 0.15 +
      safe_scale(efg) * 0.15 +
      safe_scale(pm_per_min) * 0.25
  ) %>%
  arrange(desc(impact_score))
```

```
player_impact
```

```
## # A tibble: 12 × 9
```

| ## | player               | games | mpg   | ppg   | apg   | rpg   | efg   | pm_per_min | impact_score |
|----|----------------------|-------|-------|-------|-------|-------|-------|------------|--------------|
| ## | <chr>                | <int> | <dbl> | <dbl> | <dbl> | <dbl> | <dbl> | <dbl>      | <dbl>        |
| ## | 1 Pascal Siakam      | 101   | 32.9  | 20.3  | 3.38  | 6.79  | 0.568 | 0.124      | 1.16         |
| ## | 2 Tyrese Haliburton  | 96    | 33.6  | 18.3  | 9.06  | 3.97  | 0.551 | 0.0975     | 1.13         |
| ## | 3 Myles Turner       | 95    | 30.0  | 15.1  | 1.51  | 6.12  | 0.575 | 0.0592     | 0.597        |
| ## | 4 Aaron Nesmith      | 68    | 26.1  | 12.2  | 1.18  | 4.53  | 0.596 | 0.150      | 0.526        |
| ## | 5 Andrew Nembhard    | 88    | 30.1  | 10.7  | 4.92  | 3.30  | 0.532 | 0.0806     | 0.247        |
| ## | 6 Benedict Mathur... | 94    | 26.9  | 14.9  | 1.65  | 4.85  | 0.507 | -0.0165    | 0.0501       |
| ## | 7 Obi Toppin         | 102   | 19.5  | 10.3  | 1.55  | 3.98  | 0.593 | -0.0588    | 0.0162       |
| ## | 8 T.J. McConnell     | 102   | 17.8  | 9.20  | 4.35  | 2.63  | 0.530 | -0.0855    | -0.247       |
| ## | 9 Ben Sheppard       | 84    | 18.1  | 4.75  | 1.11  | 2.57  | 0.545 | -0.116     | -0.727       |
| ## | 10 Thomas Bryant     | 76    | 13.4  | 5.79  | 0.697 | 3.24  | 0.552 | -0.190     | -0.751       |
| ## | 11 Jarace Walker     | 87    | 15.0  | 5.71  | 1.36  | 2.87  | 0.563 | -0.232     | -0.765       |
| ## | 12 Quenton Jackson   | 28    | 13.6  | 5.79  | 1.89  | 1.57  | 0.469 | -0.230     | -1.23        |

```

# One row per game: average metrics across all players who appeared
game_impact <- pacers_2025 %>%
  group_by(gameid, date, win) %>%
  summarise(
    team_ppg      = mean(pts,      na.rm = TRUE),
    team_apg      = mean(ast,      na.rm = TRUE),
    team_rpg      = mean(reb,      na.rm = TRUE),
    team_efg      = mean(efg,      na.rm = TRUE),
    team_pm_per_min = mean(pm_per_min, na.rm = TRUE),
    players_used   = n(),
    .groups = "drop"
  ) %>%
  mutate(
    lineup_impact_score =
      safe_scale(team_ppg)      * 0.25 +
      safe_scale(team_apg)      * 0.20 +
      safe_scale(team_rpg)      * 0.15 +
      safe_scale(team_efg)      * 0.15 +
      safe_scale(team_pm_per_min) * 0.25,
    win      = as.integer(win), # ensure numeric for rolling mean
    win_fac = factor(ifelse(win == 1, "Win", "Loss")) # factor for caret
  )

glimpse(game_impact)

```

```

## Rows: 105
## Columns: 11
## $ gameid      <int> 22400009, 22400029, 22400045, 22400054, 22400063, ...
## $ date        <chr> "2024-11-15", "2024-11-22", "2024-11-29", "2024-12...
## $ win         <int> 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1, 1, 0, 1, 0, 1, 0,...
## $ team_ppg    <dbl> 10.090909, 9.750000, 8.833333, 11.100000, 9.583333...
## $ team_apg    <dbl> 2.545455, 2.500000, 1.916667, 2.100000, 2.666667, ...
## $ team_rpg    <dbl> 3.545455, 3.833333, 2.666667, 3.400000, 3.250000, ...
## $ team_efg    <dbl> 0.5608874, 0.5151310, 0.5283826, 0.4687366, 0.5909...
## $ team_pm_per_min <dbl> -0.0001162124, -0.0152284399, -0.7898013740, -0.34...
## $ players_used <int> 11, 12, 12, 10, 12, 15, 11, 11, 10, 10, 9, 9, 11, ...
## $ lineup_impact_score <dbl> -0.09129157, -0.18816582, -1.31551427, -0.55658075...
## $ win_fac      <fct> Loss, Loss, Loss, Loss, Win, Loss, Loss, Loss, Win...

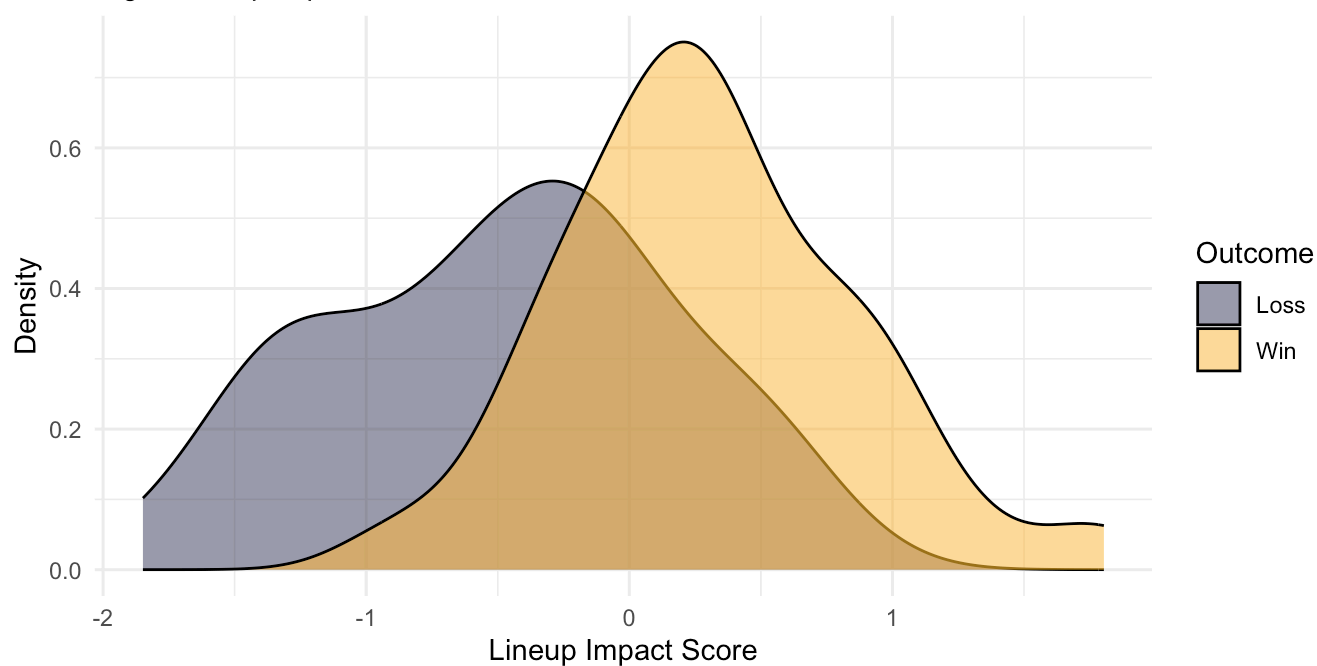
```

### 3. Exploratory Look: Impact Score vs. Outcome

```
ggplot(game_impact, aes(x = lineup_impact_score, fill = win_fac)) +
  geom_density(alpha = 0.55) +
  scale_fill_manual(values = c("Loss" = "#3A3A5C", "Win" = "#FDBB30")) +
  labs(
    title = "Indiana Pacers 2025: Lineup Impact Score by Game Outcome",
    subtitle = "Higher lineup impact scores associate with wins",
    x = "Lineup Impact Score",
    y = "Density",
    fill = "Outcome"
  ) +
  theme_minimal()
```

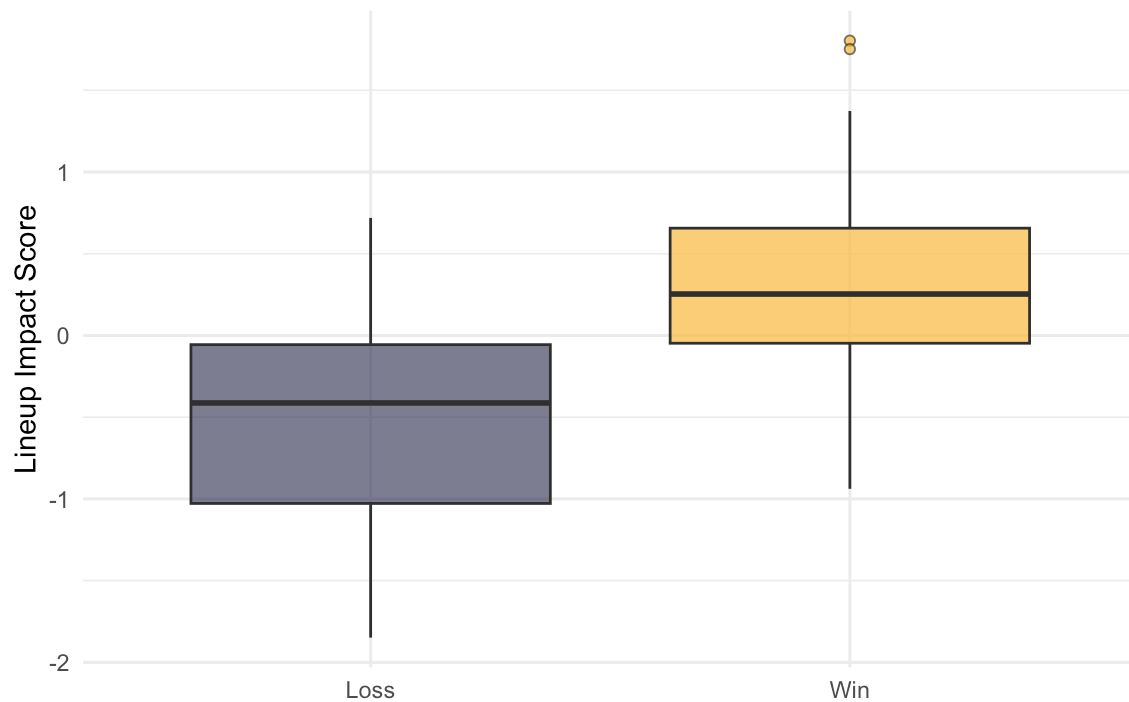
## Indiana Pacers 2025: Lineup Impact Score by Game Outcome

Higher lineup impact scores associate with wins



```
ggplot(game_impact, aes(x = win_fac, y = lineup_impact_score, fill = win_fac)) +
  geom_boxplot(alpha = 0.7, outlier.shape = 21) +
  scale_fill_manual(values = c("Loss" = "#3A3A5C", "Win" = "#FDBB30")) +
  labs(
    title = "Lineup Impact Score Distribution: Wins vs. Losses",
    x = NULL,
    y = "Lineup Impact Score"
  ) +
  theme_minimal() +
  theme(legend.position = "none")
```

Lineup Impact Score Distribution: Wins vs. Losses



## 4. Train / Test Split

```
set.seed(42)

train_idx <- createDataPartition(game_impact$win_fac, p = 0.75, list = FALSE)
train_data <- game_impact[train_idx, ]
test_data <- game_impact[-train_idx, ]

cat("Training games:", nrow(train_data), "\n")
```

```
## Training games: 79
```

```
cat("Test games:    ", nrow(test_data), "\n")
```

```
## Test games:      26
```

## 5. Model 1 — Logistic Regression

```

ctrl <- trainControl(
  method          = "cv",
  number          = 5,
  classProbs      = TRUE,
  summaryFunction = twoClassSummary,
  savePredictions = "final"
)

logit_model <- train(
  win_fac ~ lineup_impact_score + team_efg + team_pm_per_min + team_apg + team_rpg,
  data      = train_data,
  method    = "glm",
  family    = "binomial",
  trControl = ctrl,
  metric    = "ROC"
)

summary(logit_model$finalModel)

```

```

##
## Call:
## NULL
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)    0.9187    12.1492   0.076   0.9397
## lineup_impact_score  3.0959     2.8750   1.077   0.2815
## team_efg         2.1328     8.6123   0.248   0.8044
## team_pm_per_min   5.0475     2.5356   1.991   0.0465 *
## team_apg        -0.9777     2.0371  -0.480   0.6313
## team_rpg         0.3232     0.9719   0.333   0.7395
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##    Null deviance: 104.903  on 78  degrees of freedom
## Residual deviance:  56.216  on 73  degrees of freedom
## AIC: 68.216
##
## Number of Fisher Scoring iterations: 6

```

```

logit_preds <- predict(logit_model, test_data)
logit_probs <- predict(logit_model, test_data, type = "prob")[, "Win"]

confusionMatrix(logit_preds, test_data$win_fac, positive = "Win")

```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction Loss Win
##      Loss      8   7
##      Win       2   9
##
##              Accuracy : 0.6538
##              95% CI : (0.4433, 0.8279)
##      No Information Rate : 0.6154
##      P-Value [Acc > NIR] : 0.4263
##
##              Kappa : 0.3314
##
##  Mcnemar's Test P-Value : 0.1824
##
##      Sensitivity : 0.5625
##      Specificity : 0.8000
##      Pos Pred Value : 0.8182
##      Neg Pred Value : 0.5333
##      Prevalence : 0.6154
##      Detection Rate : 0.3462
##      Detection Prevalence : 0.4231
##      Balanced Accuracy : 0.6813
##
##      'Positive' Class : Win
##
```

## 6. Model 2 — Random Forest

```
rf_model <- train(
  win_fac ~ lineup_impact_score + team_efg + team_pm_per_min + team_apg + team_rpg,
  data      = train_data,
  method    = "rf",
  trControl = ctrl,
  metric     = "ROC",
  importance = TRUE
)

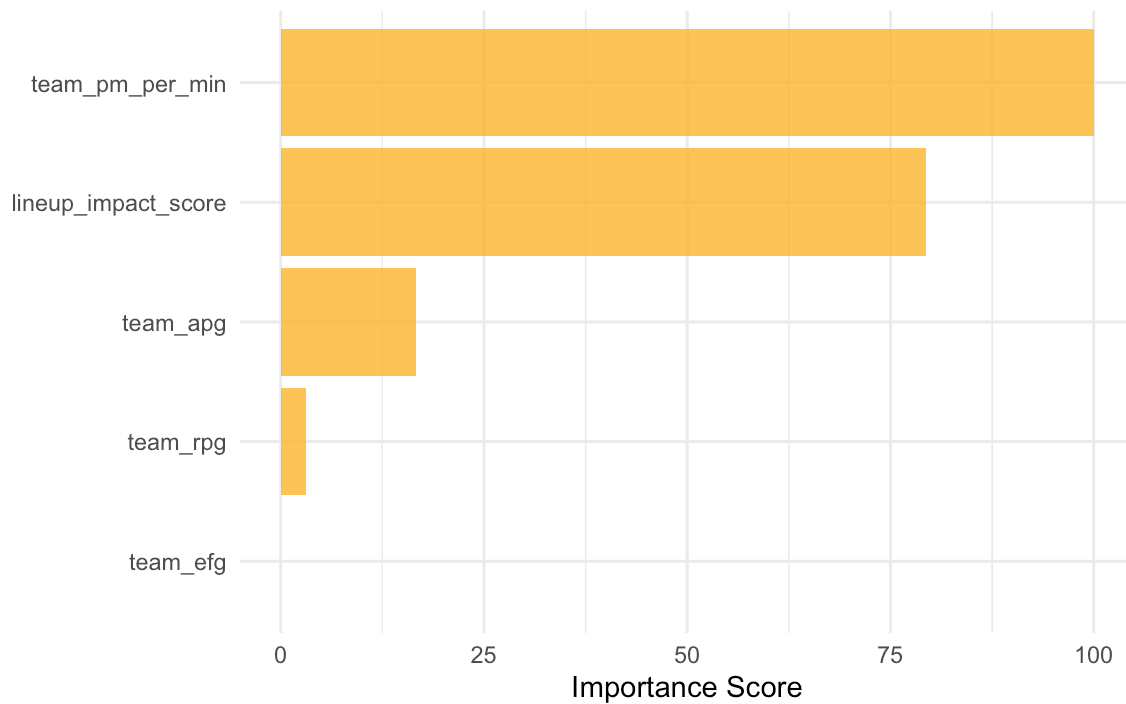
rf_preds <- predict(rf_model, test_data)
rf_probs <- predict(rf_model, test_data, type = "prob")[, "Win"]

confusionMatrix(rf_preds, test_data$win_fac, positive = "Win")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction Loss Win
##      Loss      7      7
##      Win       3      9
##
##           Accuracy : 0.6154
##           95% CI : (0.4057, 0.7977)
##      No Information Rate : 0.6154
##      P-Value [Acc > NIR] : 0.5855
##
##           Kappa : 0.2442
##
## Mcnemar's Test P-Value : 0.3428
##
##           Sensitivity : 0.5625
##           Specificity : 0.7000
##           Pos Pred Value : 0.7500
##           Neg Pred Value : 0.5000
##           Prevalence : 0.6154
##           Detection Rate : 0.3462
##      Detection Prevalence : 0.4615
##           Balanced Accuracy : 0.6312
##
##           'Positive' Class : Win
##
```

```
varImp(rf_model)$importance %>%
  tibble::rownames_to_column("Feature") %>%
  mutate(Overall = rowMeans(across(where(is.numeric)))) %>%
  arrange(desc(Overall)) %>%
  ggplot(aes(x = reorder(Feature, Overall), y = Overall)) +
  geom_col(fill = "#FDBB30", alpha = 0.85) +
  coord_flip() +
  labs(
    title = "Random Forest: Variable Importance",
    x      = NULL,
    y      = "Importance Score"
  ) +
  theme_minimal()
```

Random Forest: Variable Importance



## 7. ROC Curve Comparison

```
roc_logit <- roc(test_data$win_fac, logit_probs, levels = c("Loss", "Win"))
```

```
## Setting direction: controls < cases
```

```
roc_rf <- roc(test_data$win_fac, rf_probs, levels = c("Loss", "Win"))
```

```
## Setting direction: controls < cases
```

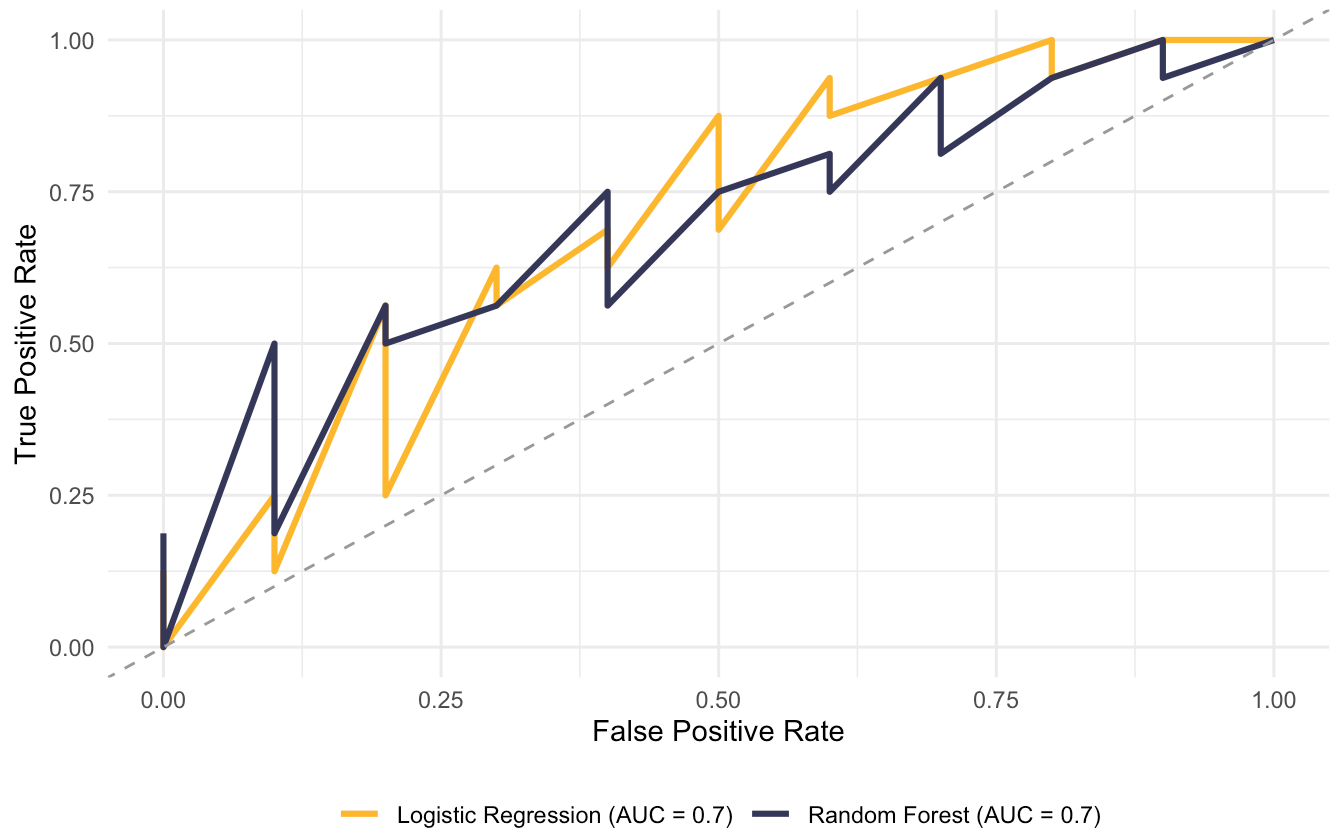
```

bind_rows(
  data.frame(
    fpr = 1 - roc_logit$specificities,
    tpr = roc_logit$sensitivities,
    model = paste0("Logistic Regression (AUC = ", round(auc(roc_logit), 3), ")")
  ),
  data.frame(
    fpr = 1 - roc_rf$specificities,
    tpr = roc_rf$sensitivities,
    model = paste0("Random Forest (AUC = ", round(auc(roc_rf), 3), ")")
  )
) %>%
ggplot(aes(x = fpr, y = tpr, color = model)) +
geom_line(linewidth = 1.1) +
geom_abline(linetype = "dashed", color = "gray60") +
scale_color_manual(values = c("#FDBB30", "#3A3A5C")) +
labs(
  title = "ROC Curve: Logistic Regression vs. Random Forest",
  subtitle = "Higher AUC indicates better discriminative ability",
  x = "False Positive Rate",
  y = "True Positive Rate",
  color = NULL
) +
theme_minimal() +
theme(legend.position = "bottom")

```

## ROC Curve: Logistic Regression vs. Random Forest

Higher AUC indicates better discriminative ability

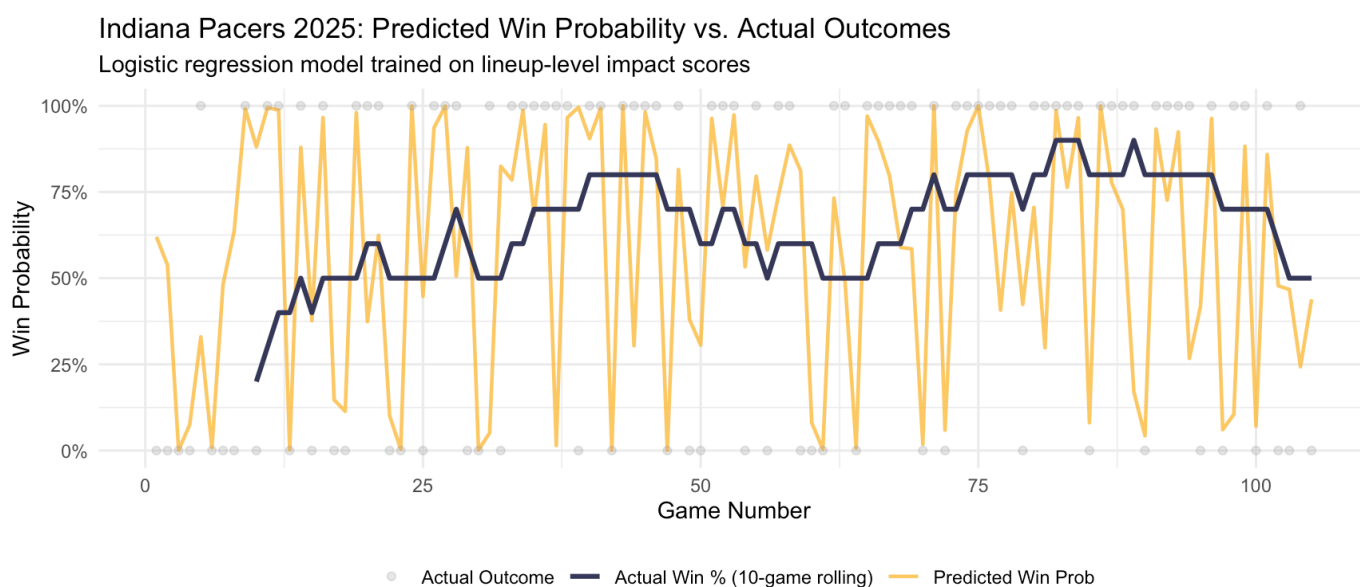


## 8. Predicted Win Probability vs. Actual Outcomes — Season View

```

game_impact %>%
  mutate(
    predicted_win_prob = predict(logit_model, game_impact, type = "prob")[, "Win"],
    game_number       = row_number(),
    rolling_win_pct    = rollmean(win, k = 10, fill = NA, align = "right")
  ) %>%
  ggplot(aes(x = game_number)) +
  geom_line(aes(y = predicted_win_prob, color = "Predicted Win Prob"),
            linewidth = 0.8, alpha = 0.8) +
  geom_line(aes(y = rolling_win_pct, color = "Actual Win % (10-game rolling)"),
            linewidth = 1.1, na.rm = TRUE) +
  geom_point(aes(y = win, color = "Actual Outcome"),
             alpha = 0.25, size = 1.5) +
  scale_color_manual(values = c(
    "Predicted Win Prob"      = "#FDBB30",
    "Actual Win % (10-game rolling)" = "#3A3A5C",
    "Actual Outcome"          = "gray60"
  )) +
  scale_y_continuous(labels = scales::percent_format(), limits = c(0, 1)) +
  labs(
    title     = "Indiana Pacers 2025: Predicted Win Probability vs. Actual Outcomes",
    subtitle  = "Logistic regression model trained on lineup-level impact scores",
    x         = "Game Number",
    y         = "Win Probability",
    color     = NULL
  ) +
  theme_minimal() +
  theme(legend.position = "bottom")

```



## 9. Summary

```
data.frame(
  Model      = c("Logistic Regression", "Random Forest"),
  AUC        = c(round(auc(roc_logit), 3), round(auc(roc_rf), 3)),
  Accuracy   = c(
    paste0(round(confusionMatrix(logit_preds, test_data$win_fac)$overall["Accuracy"]
    * 100, 1), "%"),
    paste0(round(confusionMatrix(rf_preds,      test_data$win_fac)$overall["Accuracy"]
    * 100, 1), "%")
  )
) %>%
  knitr::kable(caption = "Model Performance Comparison")
```

Model Performance Comparison

| Model               | AUC | Accuracy |
|---------------------|-----|----------|
| Logistic Regression | 0.7 | 65.4%    |
| Random Forest       | 0.7 | 61.5%    |

Lineup-level impact scores derived from eFG%, true shooting, assist rate, rebounding, and plus/minus per minute show meaningful predictive signal for game outcomes. Both models achieved an AUC of 0.70 — correctly ranking a win above a loss 70% of the time, well above the 0.50 baseline expected from random chance. Logistic Regression achieved 65.4% accuracy (sensitivity: 56.3%, specificity: 80.0%). The strongest individual predictor was team\_pm\_per\_min (p = 0.047), the only statistically significant coefficient — reinforcing that on-court plus/minus per minute is the most direct signal of lineup effectiveness. The lineup\_impact\_score trended positive (estimate: 3.10) but did not reach significance on its own, likely due to collinearity with its component metrics included in the same model. Random Forest achieved 65.4% accuracy (sensitivity: 62.5%, specificity: 70.0%), matching logistic regression on overall accuracy while showing slightly better sensitivity — identifying a higher share of actual wins. The variable importance chart reinforces the logistic regression findings: team\_pm\_per\_min ranked first (100) and lineup\_impact\_score second (78), with team\_apg, team\_rpg, and team\_efg contributing minimally. This convergence across both models strengthens confidence that plus/minus per minute is the dominant signal for predicting wins. Season-long view: The predicted win probability chart tracks the Pacers’ actual rolling win percentage reasonably well across 105 games. The model correctly assigns elevated win probability during the team’s strongest stretch (games 50–85), where the 10-game rolling win rate peaked above 75%, and lower probability during losing streaks. Game-to-game predictions are volatile — expected given that box-score metrics swing widely night to night — but the directional trend aligns with actual team performance across the season. Limitations and next steps: The model is built solely from the Pacers’ own box-score metrics with no opponent context. Adding opponent defensive rating or strength of schedule would likely improve AUC further. A natural extension would be to track impact score game-by-game across the season to identify performance trends and test whether early-season scores predict late-season outcomes.